

***Factorial Time complexity
using Recursion Tree Method***

$$T(n) = \begin{cases} 1 & , \text{for } (n = 0) \\ 1 & , \text{for } (n = 1) \\ T(n - 1) + T(1) & , \text{for } (n > 1) \end{cases}$$

Or for

$$T(n) = \begin{cases} 1 & , \text{for } (n = 0) \\ 1 & , \text{for } (n = 1) \\ T(n - 1) + 1 & , \text{for } (n > 0) \end{cases}$$

Lets recall the program:

```
int factorial(int n)
{
    // Base case: if n is 0 or 1, return 1
    // This is because the factorial of 0 and 1 is 1
    if (n == 0 || n == 1)
    {
        return 1;
    }
}
```

```

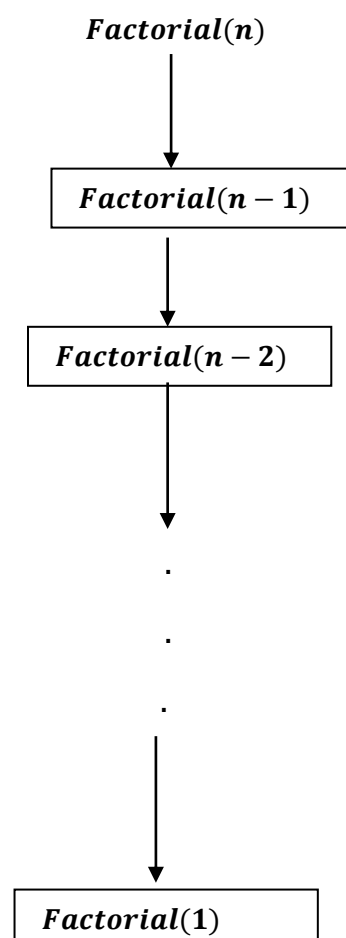
else
{
    // Recursive case: n times factorial of n-1
    // This is the definition of factorial: the product
    of an integer and all the integers below it
    return n * factorial(n - 1);
}
}

```

The Work per Problem: While it waits for $\text{factorial}(n - 1)$ to finish, what physical work does the CPU have to do? It just has to multiply the result by n . Multiplying two numbers takes a tiny, fixed amount of CPU time. Because there are no loops, this work is constant time: $O(1)$, which we call 'C'.

Therefore, the Work per Problem is Constant (C).

And recursion goes like :



Level	No. of problems	Problem Size	Work /Cost Per Problem (Multiply)	Work done = Work per Problem × No. of Problems
0	1	n	C	$1 \times C = C$
1	1	$n - 1$	C	$1 \times C = C$
2	1	$n - 2$	C	$1 \times C = C$
.	.	.	.	.
$n - 1$	1	1 (Base Case)	$T(1)$	$1 \times T(1) = T(1)$

$Total\ Cost = C + C + C + \dots (N - 1)times + T(1).$

$$= C(N - 1) + T(1)$$

**We know , $T(1)$ is 1 instead lets write $T(1) = C_1$,
representing constant.**

$$= C(N - 1) + C_1$$

As, C and C_1 both are constant Applying Big O , we get:

$$O(N - 1)$$

$$= O(N)$$
